Comprehensive Report of the TARA Processor

CS25B031 - Hemant K. Ujjwal
CS25B067 - Praket Thakur

Contents

1. Processor overview
2. Control unit
3. Datapath
4. Structural components
5. Instruction set
6. Control word and microinstructions
7. Instruction fetch and program counter
8. Memory data register and memory
9. Architectural liberties and implementation details
10. Conclusion

1. Processor overview

TARA is a 16-bit processor implemented as a Logisim circuit. The design is organized around a datapath and a control unit. The datapath contains the registers, ALU, register file and memory interface, while the control unit sequences operations by reading microinstructions from a control ROM. A shared central bus connects the major datapath components, and the microcode controls which component drives the bus and which registers or memory elements are loaded.

The implementation supports arithmetic and logical operations, immediate-value operations, word and byte memory access, conditional and unconditional control flow, subroutine calls and returns, and stack operations. The following sections describe the hardware organization and the implementation choices used to support these instructions.

Control Unit :

It dictates the operations occurring in the datapath and for the overall control of the operations.

The major components are listed below :

1. **Control ROM :**

The read-only memory stores the microcode. Its 11-bit address is composed of the current conditional flags, the instruction opcode and the T-state: Z is bit 10, N is bit 9, the 5-bit opcode occupies bits 8–4, and the 4-bit T-state occupies bits 3–0. This gives 2048 possible control-ROM addresses.

2. **T-state counter :**

A counter that starts at 0 and keeps track of the current part of the instruction execution cycle. It is not a ring counter because some instructions require more execution cycles than others. The counter is returned to 0 by the step_reset control signal.

The Datapath :

The highest level of abstraction for the cpu.

Components :
Registers :

1. PC : The program counter. Responsible for keeping track of the current instruction being operated. Takes as input :
    a. D : 16 bit input pin. Driven through the bus.
    b. CLK
    c. Load : Write enable for PC. The counter only updates if load is 1
    d. RST
    e. Increment : If 0, data is loaded from the D input. If 1. It increments the PC based on the amount decided by the Add pin using an inbuilt adder
    f. Rel11 : 16 bit input which can be added to the counter
    g. Add : Decides weather to add an arbitrary amount to the PC or the default 2

Outputs to the central bus.

2. MAR : The memory address register. Takes in input directly from the central bus. Output is connected directly to the Ram_Word's address pin, considering it is specifically only used for memory access.
3. MDR : Stores the data that is coordinated with the RAM.
   Takes in input :
   a. Bus_Out : Input connected to the bus via a controlled buffer
   b. Ram_Out : Input from the Ram at address determined by the MAR
   c. MDR_select : Selects between the Bus and RAM inputs.
   d. Clk, Load, RST
4. IR : The instruction register which feeds the decoder. Takes input from the bus directly.
5. alu_acc : A buffer register to store the value from the value. Responsible for writing alu_out to the bus and the controlled buffer is controlled by the same alu_out pin.
6. status_reg : Combinational unit takes in input from alu and sets the various flags.
7. regfile registers :
   R0-R5 : General purpose registers.
   R6 : Link register - Stores the return address when entering CALL and outputs it to pc to return to correct point of execution when it is RET
   R7 : Stack pointer - Points to the top of the stack. Grows downwards.

Structural Components :

1. alu : The 16 bit alu made in a previous take home assignment.
2. regfile : A file with 8 registers, each of 16 bits.
3. ram_word : A 64KB ram.
4. tara_decoder :
   The unit responsible for parsing the instruction loaded into IR.
   Among its functionality are the following -
   a. Parsing opcode, imm8, off5, rel8, rel11 from the taken instruction
   b. Using the opcode, the format classifications of the inputs can be done. This is obtained by using a multiplexer to map different opcodes to formats.
   c. The most important portion is the implementation of the three regfile addresses - rd, ra, rb; responsible for determining what registers are written to and read from using the formats and opcodes.
      RD - Default : Bits 8-10 | opcode 17-18 or 26 : 6 | 26 > opcode > 18 : 7
      RA - Default : Bits 5-7 | Format 3-7 or opcode 24(ret) : RD
      RB - Default : Bits 2-4 | Format 4 : Bits 5-7

Instruction set overview

The opcode is 5 bits wide. The implemented instruction set, as represented in the control-ROM generation script, is summarized below. Opcode 01100 is not assigned in the current microcode.

Opcode (binary)	Opcode (decimal)	Instruction
00000	0	NOP
00001	1	HLT
00010	2	MOV
00011	3	LIL
00100	4	LIH
00101	5	LDW
00110	6	STW
00111	7	LDB
01000	8	STB
01001	9	ADD
01010	10	SUB
01011	11	ADDI
01101	13	AND
01110	14	OR
01111	15	XOR
10000	16	NOT
10001	17	SHL
10010	18	SHR
10011	19	SLT
10100	20	BZ
10101	21	BN
10110	22	JMP
10111	23	CALL
11000	24	RET
11001	25	PUSH
11010	26	POP

The data-processing group includes ADD, SUB, ADDI, AND, OR, XOR, NOT, SHL, SHR and SLT. MOV transfers a register value, while LIL and LIH load the lower and higher immediate portions. LDW and STW operate on words; LDB and STB operate on bytes. BZ and BN are conditional branches, JMP is an unconditional jump, CALL and RET implement subroutine control flow, and PUSH and POP implement stack access.

List of datapath internal flags :

is_branch - set for BZ/BN
is_shift - set for SHL/SHR
is_addi - set for ADDI

is_store - set for STB and STW

is_stack - set for PUSH and POP

is_push - set for PUSH

is_pop - set for POP
is_lors - is a load or store instruction
is_byte - is an instruction that deals with byte sized objects instead of word sized ones
leastsigopcode - the least significant bit of the opcode
addr_parity - The parity of the address set using MAR
is_7 - to select 7 over RD from the decoder
is_2 - to feed 2 to the alu instead of the usual data from regfile

imm8 - sign extended imm8

off5 - sign extended off5
Z_temp - temporary Z flag used for BZ. Set by comparing with 0

N_temp - temporary N flag used for BN. Set by comparing with 0

lnots - Is MDR load but not select, i.e. MDR reads from BUS and not RAM

Control word and Microinstructions :

The Microinstruction composition:

Bit 19-22 : Bus_source - Chooses what drives the central bus and a decoder to enable that controlled buffer, preventing any bus contentation. The enable pin is always set to 1 because reading is done carefully to strictly avoid any unintended data writes.
The bus_source uses 4 bits to choose amongst the following 10 outputs :

1. 0b0000 - PC
2. 0b0001 - MDR
3. 0b0010 - IR
4. 0b0011 - regfile, specifically the register defined by address RA
5. 0b0100 - imm8, zero extended
6. 0b0101 - off5, sign extended
7. 0b0110 - rel8, sign extended
8. 0b0111 - rel11, sign extended
9. 0b1000 - alu_out, from alu_acc register which reads from alu
10. 0b1001 - LIH_IN, output of a sub-circuit made specifically to accommodate the LIH instruction by preserving the lower byte while writing from imm8 to the higher byte.

Bit 18 : PC_load - Permits updating of PC

Bit 17 : PC_increment - Determines whether to load from the data pin or increment current value of the register.

Bit 16 : PC_add - Determines whether to increment the PC by the default 2 or add an amount determined by the Rel11 input

Bit 15 : MAR_load

Bit 14 : MDR_select - Internally selects between reading input from Bus(0) or Ram(1)

Bit 13 : MDR_load

Bit 12 : IR_load

Bit 11 : Alu_out - Loads the alu output into the accumulator

Bit 10 : Regfile_load

Bit 9 : Mem_read - Asynchronous read from address determined by MAR

Bit 8 : Mem_write

Bit 7 : Status_load - Latches the conditional flags

Bit 6 : Step_reset - Resets the T-counter to 0

Bit 5 : Halt - Halts the execution of the program

Bit 0-4 : Alu_select - Select what operation the alu computes

The fetch sequence :

T0 : PC -> MAR
T1 : Mem[MAR] -> MDR

T2 : MDR -> IR

Note : T1 does not increment the PC to ready it for the next instruction.

Instruction execution sequence

Every instruction begins with the same three-state fetch sequence: T0 transfers the PC to MAR, T1 reads the memory word addressed by MAR into MDR, and T2 transfers MDR into IR. The PC is deliberately not incremented during this fetch sequence. The instruction is then decoded, and its execution microinstructions begin at the subsequent T-states.

The control ROM selects a microinstruction using the opcode, the current T-state and the Z/N conditions. A microinstruction can select a bus source, load a register, access memory, perform an ALU operation, update status flags, reset the T-state counter or halt execution. Instructions normally finish by asserting step_reset, returning control to the fetch sequence.

The Program Counter

The PC stores the address of the instruction currently being executed and only takes even values, since instructions are word-aligned. Unlike a conventional fetch sequence that increments the PC in advance, this implementation leaves the PC unchanged during fetch. This avoids complications when instructions calculate a new PC value, especially relative branches and jumps.

Instead, the PC is incremented as part of the instruction's final micro-operations, together with step_reset. The step_reset signal is synchronous, so the T-state counter resets on the next rising clock edge. This gives the microcode greater freedom when calculating the PC without adding extra clock cycles to the instruction.

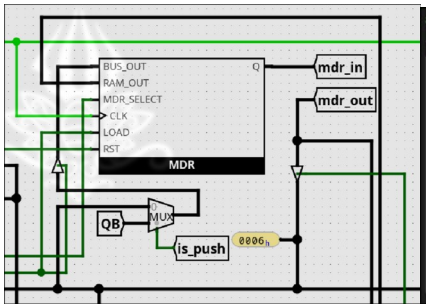
The fetch sequence

An example instruction

0x40 is parsed by the control unit to set set_reset.
0x60000 corresponds to PC_load + PC_increment.
0x60040 combines their functions

Note : Also the instructions involving addition of PC by rel11 instead involve twice of the rel11 due to the nature of ISA. This is done by adding two splitters to feed rel11<<1 instead of rel11.

The Memory Data Register :
Stores the data that is coordinated with the RAM. Both its input and output have tweaks.




```
#PUSH : opcode : 11001
for i in range(4):
    add = (1<<9)
    add = add*i
    add3 = add + 3 + (1<<8) + (1<<7) + (1<<4)
    add4 = add + 4 + (1<<8) + (1<<7) + (1<<4)
    rom_hex[add3] = 0x40080a #r7 decrement
    rom_hex[add4] = 0x400c00
    rom_hex[add4+1] = 0x188000 #r7 -> mar
    rom_hex[add4+2] = 0x002000 #mdr load. Directly through QB, hardcoded.
    rom_hex[add4+3] = 0x080100 #mdr -> mem[mar]
    rom_hex[add4+4] = 0x60040
```

Input :

Ram_out takes input directly from the RAM_write which is asynchronous.
Bus_out is controlled using MDR_load and takes input from the central Bus.
But there is one adjustment :
In push instructions, based on the ISA format we have “opcode | rstk | 00000000”
But for the instruction itself we need to decrement R7, while also inputting rstk’s data to the ram based on the data of the decremented R7.
To do the tara_decoder has orientation so that it outputs : rd = ra = 7, rb = rstk
Using rd, ra and certain conditional flags we can decrement R7.
But accessing rstk remains the issue, considering rb’s output doesn’t connect to the main bus. So to avoid redoing the circuit, we instead added a condition to directly chose QB, the data of rb, when it is a push instruction.

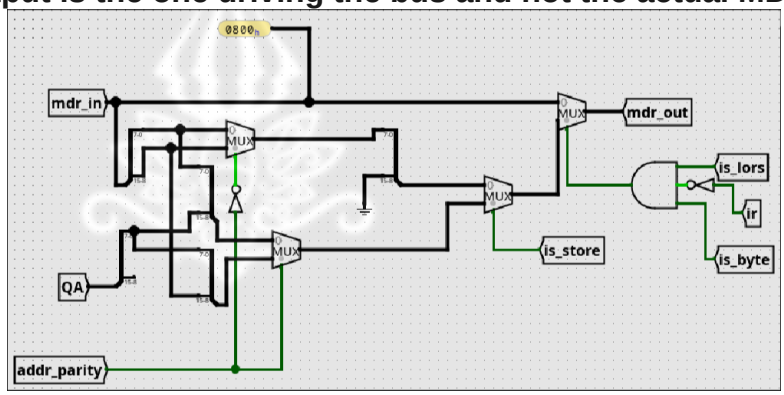
Output :

The MDR output :

In different instructions MDR can write to memory or from memory. Also the data to be written might not be the data fed into MDR. This happens when the instructions are concerned with byte sized objects.

This is solved by adding a sub_circuits which takes the value in MDR currently and based on the state chooses what to output.

This output is the one driving the bus and not the actual MDR value.



Unless it’s LDB or STB it is directly mdr_in.

Note : The negation of ir, is described in the next point of this section.

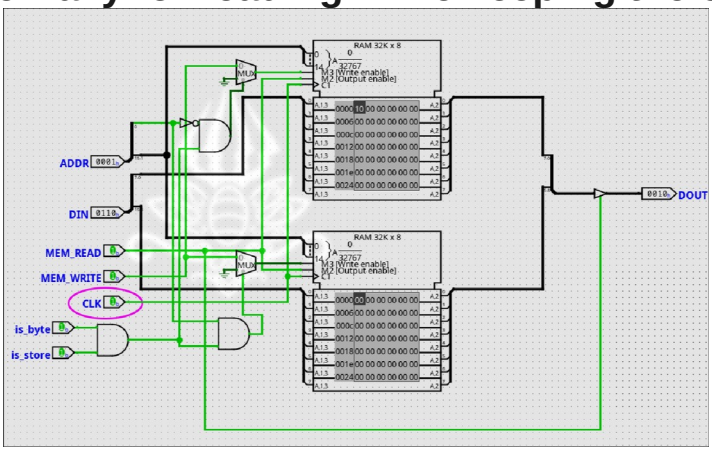
Since MDR always loads the whole word, this uses the lower or higher byte of data, based on the parity of the MAR’s address. Then extends it to a word, using either 0s or by something else.

(Note : Earlier it was used to preserve content, but later it was rendered obsolete by as the ram_word itself was used to chose internally to handle this)

The ram_word :

It consists of two separate ram objects. One corresponds to even address and other to odd address bytes. Writing of words is always done on even addresses. With synchronous write and asynchronous read.

Specifically it has two extra pins : is_store, is_bytes. They are responsible for the STB functionality and selecting one of those rams internally for reading while keeping the other off.



Due to the nature of the RAM addressing and the fact that they only use the more significant 15 bits of the address for their address, it means that the address is right shifted by 1 when saved into the RAM, i.e. half of that value.

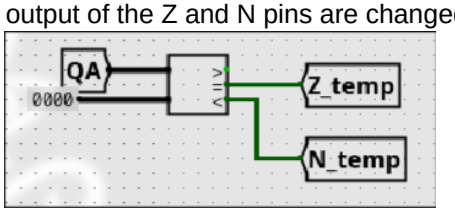
Stack and subroutine conventions

R7 is used as the stack pointer and the stack grows towards lower addresses. PUSH decrements R7 before storing the selected register value at the new stack location. POP reads the value at the address held by R7, writes it to the destination register, and then increments R7. R6 is the link register: CALL saves the return address in R6, and RET transfers the saved address back to the PC. This is a single link-register convention, so nested subroutine calls require the caller to preserve R6 if it must retain an earlier return address.

The architectural liberties :

This section lists all the unique implementations used under the top layer of abstraction.

1. The Branch instructions BZ, BN do not read from the status_reg flags. Instead a separate subcircuit is used that compares the selected register's data with 0 to populate these bits and then for these particular instructions the output of the Z and N pins are changed to these faux Z and N.



2. BZ, BN are inefficient :

```
#BZ : opcode : 10100
for i in range(4):
    add = (1<<9)
    add = add*i
    add3 = add + 3 + (1<<8) + (1<<6)
    add4 = add + 4 + (1<<8) + (1<<6)
    rom_hex[add3] = 0x400809 #set status reg
    if i < 2 :
        rom_hex[add4] = 0x060000 #pc += 2
        rom_hex[add4+1] = 0x40
    else :
        rom_hex[add4] = 0x070000 #add pc
        rom_hex[add4+1] = 0x060040
```

```
#BN : opcode : 10101
for i in range(4):
    add = (1<<9)
    add = add*i
    add3 = add + 3 + (1<<8) + (1<<6) + (1<<4)
    add4 = add + 4 + (1<<8) + (1<<6) + (1<<4)
    rom_hex[add3] = 0x400809 #set status reg
    if i&1 == 0 :
        rom_hex[add4] = 0x060000 #pc += 2
        rom_hex[add4+1] = 0x40
    else :
        rom_hex[add4] = 0x070000 #add pc
        rom_hex[add4+1] = 0x060040
```

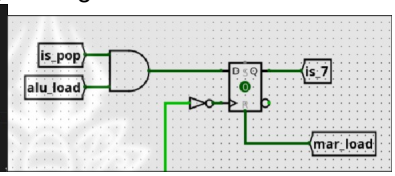
For the cases where the branching is not being done as the conditions are not met, we are incrementing the PC right before calling step_reset as we do in usual instructions but instead of doing it in 1 instruction we do it in 2 seperate steps.

This is a deliberate attempt to keep the number of clock cycles for the instruction constant irrespective of the internal state.

3. LIH_IN : A separate sub-circuit that drives the bus specifically for the LIH instruction using the direct input from RD's data and the imm8 to obtain accurate implementation and avoid any loss of data.

4. The POP instruction : The only instruction involving potentially writing to two addresses of the regfile.

```
#POP : opcode : 11010
for i in range(4):
    add = (1<<9)
    add = add*i
    add3 = add + 3 + (1<<8) + (1<<7) + (1<<5)
    add4 = add + 4 + (1<<8) + (1<<7) + (1<<5)
    rom_hex[add3] = 0x188000 #r7 -> mar
    rom_hex[add4] = 0x086200 #mem[mar] -> mdr
    rom_hex[add4+1] = 0x080400 #mdr -> reg_file
#### Note : On falling edge of the clock cycle and when alu_out is set, the RD is changed to 7 using a flip flop for holding
    rom_hex[add4+2] = 0x400809 #r7 increment
    rom_hex[add4+3] = 0x400c00
    rom_hex[add4+4] = 0x60040
#### Flip flop is reset next time mar is loaded btw. i.e. first step of next fetch cycle
```



To accommodate the writing of the popped value of the stack into RD first and then R7 being incremented as the stack grows upwards, we use a D flip-flop.

The flip-flop drives a flag 'is_7', which selects 7 over the RD derived by the tara_decoder into the regfile.

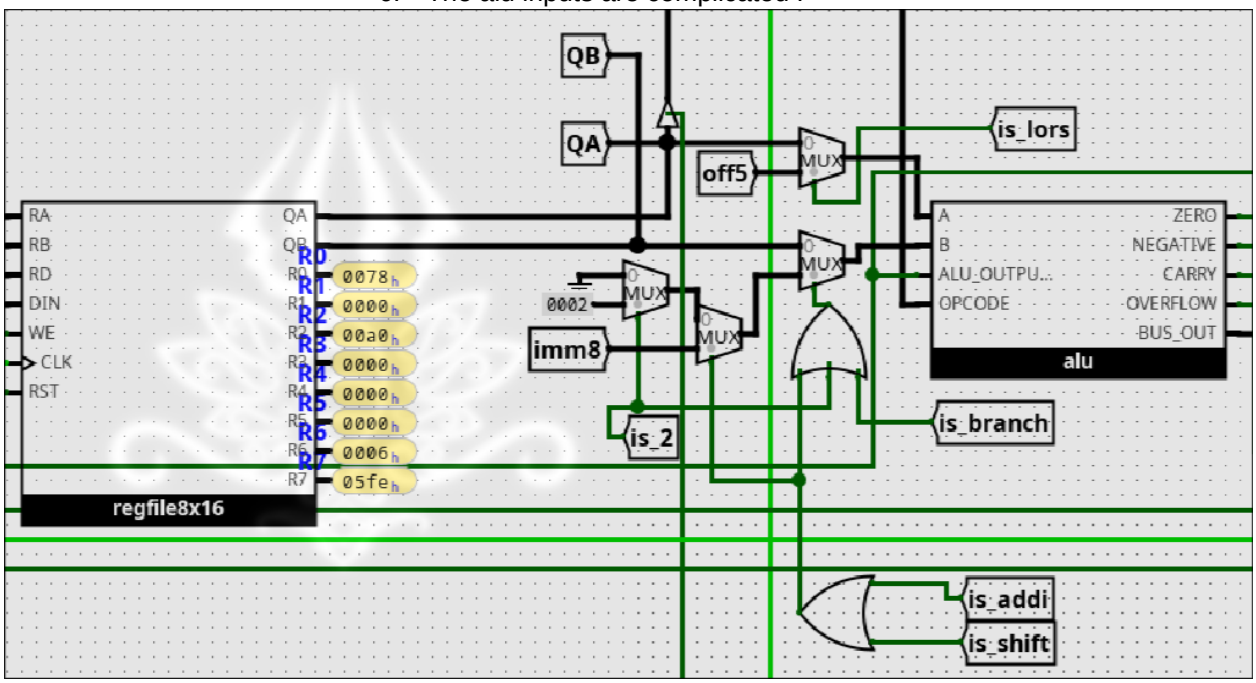
It is asserted on the falling edge between T5 and T6 of the POP instruction, specifically when the alu_load is set.

This then is used as a selector in a mux that when set chooses 7 otherwise the RD parsed by the decoder.

At the start of the next fetch sequence, i.e. when mar_load is set, it resets to prevent any unintended behavior.

5. Various computational flags are used internally to facilitate the implementation of particular instructions. This is convenient, fast, and we preferred that over extensive bus or register usage.
   The ~ir into the AND when choosing mdr_out :
   During the fetch sequence the loaded opcode is that of the previous instruction. As such if it were to contaminate the mdr_out it would ruin the loading of instructions.

6. The alu inputs are complicated :



a. When it is a load/store instruction, is_lors is set.
Based on the ISA, these instructions are of the format “opcode|rdata|rbase|offset”.
Hence we need to compute rbase + offset and then feed this into the MAR. But we also need to load the data of rdata into MDR. Because only QA can drive the bus, and we need the output of two registers. Here we use the idea of using rdata as RA and rbase as RB. Then offset is used instead of QA inside the ALU. The ALU_ACC writes to MAR, when regfile can write to MDR.

b. BZ/BN in an old version used QA+0 to load Z and N. That has been made obsolete now, but the hardware still persists.

c. For CALL, JMP, PUSH, POP :
These use increment/decrement by 2, and as such it is useful to feed 2 directly to the ALU via valid multiplexer arrangement.

d. ADDI or SHL/SHR :
These use ALU operations but only one register as input, the other input is the sign extended imm8. So this is also provided through multiplexers.

Conclusion

The TARA processor brings together the datapath, control logic, memory interface and microcode into a working 16-bit processor implementation. The design uses a shared bus and a control ROM to sequence instruction execution, with additional internal flags and small supporting circuits where the ISA creates special cases. The implementation choices described in this report were made to keep the datapath manageable while supporting the required instruction set, including memory, branching, subroutine and stack operations.